Blocking vs Non-blocking 할당과 실행 순서(answer)

- 다음 두 always 블록이 실행되었을 때, 클럭(clk)의 상승 에지 이후 reg a와 reg b에 최종적으로 저장되는 값을 각각 구하고, 그 이유를 설명하시오. (가정: 초기값은 a = 0, b = 0 이다.)
- Case A (Blocking Assignment =)

```
always @(posedge clk) begin
    a = 10;
    b = a;
end
```

- Task를 instantiate하고, a, b, c의 waveform을 받아 검토한다.

```
always @(posedge clk) begin
    a <= 10;
    b <= a;
end
```

- **Case A (Blocking): a = 10, b = 10**
  - 이유: Blocking 할당(=)은 순차적으로 실행됩니다. 윗줄 a = 10이 먼저 실행되어 a가 10으로 업데이트된 후, 아랫줄 b = a가 실행되므로 b에는 업데이트된 a의 값(10)이 들어갑니다.
- **Case B (Non-blocking): a = 10, b = 0**
  - 이유: Non-blocking 할당(<=)은 동시(concurrent)에 평가됩니다. 우변의 값들을 먼저 계산해두고(Evaluation), 블록이 끝날 때 좌변에 업데이트합니다. b <= a가 실행될 시점의 a는 아직 업데이트 전의 값(초기값 0)이므로 b에는 0이 들어갑니다.

Fork-Join_none 실행 순서와 시간 추론(answer)

- 다음 Verilog 코드를 분석하여, 시뮬레이션 실행 시 콘솔에 출력되는 메시지의 순서와 각 메시지가 출력되는 **시뮬레이션 시간($time)**을 쓰시오. (가정: timescale 1ns/1ns로 설정되어 있다.)

```
module test_fork;
    initial begin
        // [A] 시작 메시지
        $display("A: Start at time %0t", $time);
        fork
            // [Process 1] 20ns 지연 후 출력
            begin
                #20;
                $display("B: Process 1 at time %0t", $time);
            end
            // [Process 2] 10ns 지연 후 출력
            begin
                #10;
                $display("C: Process 2 at time %0t", $time);
            end
        join_none  // <--- 핵심 포인트
        // [D] fork 블록 직후 메시지
        $display("D: After Join at time %0t", $time);
        // [E] 메인 쓰레드 지연 15ns
        #15;
        $display("E: End Code at time %0t", $time);
    end
endmodule
```

1.__ns : (어떤 메시지?)
2.__ns : (어떤 메시지?)
3.__ns : (어떤 메시지?)
4.__ns : (어떤 메시지?)
5.__ns : (어떤 메시지?)

1. 0ns : A: Start...
2. 0ns : D: After Join...
3. 10ns : C: Process 2...
4. 15ns : E: End Code...
5. 20ns : B: Process 1...

Pattern generator task 만들기

- 배경: logic simulation 을 할 때 input 으로 펄스열(sequence)을 주게 되는데, initial 문안에서 #n; 을 사용하면 여러 개의 입력을 동시에 생각하게 될 경우 가독성이 떨어지고 코딩 작업에도 시간이 많이 소요된다.
- 요구기능: input 으로 string 변수를 선언하여 여기에 input sequence 를 주고, 이 string 변수를 가져다가 pulse 열을 만들어 내는 task를 설계한다.
- Task를 instantiate하고, a, b, c의 waveform을 받아 검토한다.

```
module test;
  bit clk, a, b, c;

  always #5 clk = ~clk;

  initial begin
    #10 a = 1;
        b = 1;
    #30 c = 1;
    //    :
    //    :
    #50 $finish;
  end

  initial begin
      $dumpvars; $dumpfile("dump.vcd");
  end
endmodule
```

```
module test;
  bit clk, a, b, c;

  always #5 clk = ~clk;

  string str_a = "01000000000";
  string str_b = "00101001000";
  string str_c = "00010000100";

  //task definition here
  //    :
  //    :
  // task instantiation here
  //    :
  initial begin
          :
    #120 $finish;
  end
          :
endmodule
```

Pattern generator task 만들기

- 배경: logic simulation 을 할 때 input 으로 펄스열(sequence)을 주게 되는데, initial 문안에서 #n; 을 사용하면 여러 개의 입력을 동시에 생각하게 될 경우 가독성이 떨어지고 코딩 작업에도 시간이 많이 소요된다.
- 요구기능: input 으로 string 변수를 선언하여 여기에 input sequence 를 주고, 이 string 변수를 가져다가 pulse 열을 만들어 내는 task를 설계한다.
- Task를 instantiate하고, a, b, c의 waveform을 받아 검토한다.

```
module test;
  bit clk, a, b, c;

  always #5 clk = ~clk;

  initial begin
    #10 a = 1;
        b = 1;
    #30 c = 1;
    //    :
    //    :
    #50 $finish;
  end

  initial begin
      $dumpvars; $dumpfile("dump.vcd");
  end
endmodule
```

```
module test;
  bit clk, a, b, c;

  always #5 clk = ~clk;

  string str_a = "01000000000";
  string str_b = "00101001000";
  string str_c = "00010000100";

  //task definition here
  //    :
  //    :
  // task instantiation here
  //    :
  initial begin
          :
    #120 $finish;
  end
          :
endmodule
```

Pattern generator task 만들기(answer)

```
module test;

  bit clk = 0;
  bit a, b, c;

  always #5 clk = ~clk;

  // 입력 시퀀스
  string str_a = "01000000000";
  string str_b = "00101001000";
  string str_c = "00010000100";

  // -------------------------------
  // Pulse Generator Task
  // -------------------------------
  task automatic drive_from_string(
      input string seq,
      output bit signal
  );
      int i;
      for (i = 0; i < seq.len(); i++) begin
          @(posedge clk);
          signal = (seq[i] == "1");
      end
  endtask
```

```
  // -------------------------------
  // Task 실행
  // -------------------------------
  initial begin
      fork
          drive_from_string(str_a, a);
          drive_from_string(str_b, b);
          drive_from_string(str_c, c);
      join
  end

  initial begin
      $dumpvars;
      $dumpfile("dump.vcd");
      #120 $finish;
  end

endmodule
```